E20288
Perera M.C.S.J.

Dataset Preparation

```
use lab06_db
```

```
db.customers.insertMany([
  { _id: 1, firstName: "John", lastName: "Doe" },
  { _id: 2, firstName: "Jane", lastName: "Smith" },
  { _id: 3, firstName: "Ali", lastName: "Khan" },
  { _id: 4, firstName: "Maria", lastName: "Garcia" },
  { _id: 5, firstName: "Sam", lastName: "Altman" },
  { _id: 6, firstName: "Elon", lastName: "Musk" },
  { _id: 7, firstName: "Steve", lastName: "Jobs" },
  { _id: 8, firstName: "Ada", lastName: "Lovelace" },
  { _id: 9, firstName: "Grace", lastName: "Hopper" },
  { _id: 10, firstName: "Alan", lastName: "Turing" }
]);
```

```
test> use lab06_db
switched to db lab06_db
lab06_db> db.customers.insertMany([
|   { _id: 1, firstName: "John", lastName: "Doe" },
|   { _id: 2, firstName: "Jane", lastName: "Smith" },
|   { _id: 3, firstName: "Ali", lastName: "Khan" },
|   { _id: 4, firstName: "Maria", lastName: "Garcia" },
|   { _id: 5, firstName: "Sam", lastName: "Altman" },
|   { _id: 6, firstName: "Elon", lastName: "Musk" },
|   { _id: 7, firstName: "Steve", lastName: "Jobs" },
|   { _id: 8, firstName: "Ada", lastName: "Lovelace" },
|   { _id: 9, firstName: "Grace", lastName: "Hopper" },
|   { _id: 10, firstName: "Alan", lastName: "Turing" }
| ]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
```

Adding Products

```
db.products.insertMany([
  { _id: 101, name: "Laptop", price: 1200 },
  { _id: 102, name: "Mouse", price: 25 },
  { _id: 103, name: "Keyboard", price: 45 },
  { _id: 104, name: "Monitor", price: 300 },
  { _id: 105, name: "USB-C Cable", price: 15 },
```

```
  { _id: 106, name: "Headphones", price: 150 },
  { _id: 107, name: "Webcam", price: 80 },
  { _id: 108, name: "RAM 16GB", price: 90 },
  { _id: 109, name: "SSD 1TB", price: 110 },
  { _id: 110, name: "GPU", price: 500 },
  { _id: 111, name: "Chair", price: 200 },
  { _id: 112, name: "Desk", price: 150 },
  { _id: 113, name: "Microphone", price: 120 },
  { _id: 114, name: "Router", price: 60 },
  { _id: 115, name: "Switch", price: 40 }
]);
```

```
lab06_db> db.products.insertMany([
|    { _id: 101, name: "Laptop", price: 1200 },
|    { _id: 102, name: "Mouse", price: 25 },
|    { _id: 103, name: "Keyboard", price: 45 },
|    { _id: 104, name: "Monitor", price: 300 },
|    { _id: 105, name: "USB-C Cable", price: 15 },
|    { _id: 106, name: "Headphones", price: 150 },
|    { _id: 107, name: "Webcam", price: 80 },
|    { _id: 108, name: "RAM 16GB", price: 90 },
|    { _id: 109, name: "SSD 1TB", price: 110 },
|    { _id: 110, name: "GPU", price: 500 },
|    { _id: 111, name: "Chair", price: 200 },
|    { _id: 112, name: "Desk", price: 150 },
|    { _id: 113, name: "Microphone", price: 120 },
|    { _id: 114, name: "Router", price: 60 },
|    { _id: 115, name: "Switch", price: 40 }
| ]);
{
  acknowledged: true,
  insertedIds: {
    '0': 101,
    '1': 102,
    '2': 103,
    '3': 104,
    '4': 105,
    '5': 106,
    '6': 107,
    '7': 108,
    '8': 109,
    '9': 110,
    '10': 111,
    '11': 112,
    '12': 113,
    '13': 114,
    '14': 115
  }
}
```

Adding Transactions

```
db.transactions.insertMany([
  { customerId: 1, productId: 101, quantity: 1, paymentMethod: "Credit Card", amount: 1200, date: ISODate("2026-02-01T10:00:00Z") },
  { customerId: 1, productId: 102, quantity: 2, paymentMethod: "Credit Card", amount: 50, date: ISODate("2026-02-01T11:30:00Z") },
```

```
  { customerId: 2, productId: 104, quantity: 1, paymentMethod: "Cash", amount: 300, date: ISODate("2026-02-01T14:00:00Z") },
  { customerId: 3, productId: 105, quantity: 10, paymentMethod: "Online", amount: 150, date: ISODate("2026-02-02T09:15:00Z") },
  { customerId: 4, productId: 110, quantity: 1, paymentMethod: "Credit Card", amount: 500, date: ISODate("2026-02-02T10:00:00Z") },
  { customerId: 5, productId: 108, quantity: 2, paymentMethod: "Cash", amount: 180, date: ISODate("2026-02-03T12:00:00Z") },
  { customerId: 6, productId: 111, quantity: 1, paymentMethod: "Credit Card", amount: 200, date: ISODate("2026-02-03T15:45:00Z") },
  { customerId: 2, productId: 103, quantity: 1, paymentMethod: "Online", amount: 45, date: ISODate("2026-02-04T08:30:00Z") },
  { customerId: 7, productId: 115, quantity: 3, paymentMethod: "Credit Card", amount: 120, date: ISODate("2026-02-04T11:00:00Z") },
  { customerId: 8, productId: 109, quantity: 1, paymentMethod: "Cash", amount: 110, date: ISODate("2026-02-05T13:20:00Z") },
  { customerId: 9, productId: 112, quantity: 1, paymentMethod: "Online", amount: 150, date: ISODate("2026-02-05T16:00:00Z") },
  { customerId: 10, productId: 107, quantity: 2, paymentMethod: "Credit Card", amount: 160, date: ISODate("2026-02-06T09:00:00Z") },
  { customerId: 1, productId: 106, quantity: 1, paymentMethod: "Cash", amount: 150, date: ISODate("2026-02-06T14:10:00Z") },
  { customerId: 3, productId: 114, quantity: 1, paymentMethod: "Online", amount: 60, date: ISODate("2026-02-07T10:30:00Z") },
  { customerId: 5, productId: 102, quantity: 1, paymentMethod: "Credit Card", amount: 25, date: ISODate("2026-02-07T15:00:00Z") },
  { customerId: 4, productId: 113, quantity: 1, paymentMethod: "Cash", amount: 120, date: ISODate("2026-02-08T11:45:00Z") },
  { customerId: 6, productId: 105, quantity: 2, paymentMethod: "Online", amount: 30, date: ISODate("2026-02-08T16:20:00Z") },
  { customerId: 7, productId: 101, quantity: 1, paymentMethod: "Credit Card", amount: 1200, date: ISODate("2026-02-09T09:15:00Z") },
  { customerId: 8, productId: 104, quantity: 2, paymentMethod: "Cash", amount: 600, date: ISODate("2026-02-10T12:40:00Z") },
  { customerId: 2, productId: 110, quantity: 1, paymentMethod: "Online", amount: 500, date: ISODate("2026-02-10T14:50:00Z") },
  { customerId: 10, productId: 108, quantity: 1, paymentMethod: "Credit Card", amount: 90, date: ISODate("2026-02-11T10:00:00Z") },
  { customerId: 9, productId: 115, quantity: 1, paymentMethod: "Cash", amount: 40, date: ISODate("2026-02-11T13:15:00Z") },
  { customerId: 1, productId: 111, quantity: 1, paymentMethod: "Online", amount: 200, date: ISODate("2026-02-12T11:00:00Z") },
  { customerId: 3, productId: 109, quantity: 2, paymentMethod: "Credit Card", amount: 220, date: ISODate("2026-02-12T15:30:00Z") },
  { customerId: 5, productId: 107, quantity: 1, paymentMethod: "Cash", amount: 80, date: ISODate("2026-02-13T09:45:00Z") },
```

```
  { customerId: 4, productId: 103, quantity: 2, paymentMethod: "Online", amount: 90, date: ISODate("2026-02-13T14:20:00Z")
},
  { customerId: 7, productId: 114, quantity: 2, paymentMethod: "Credit Card", amount: 120, date: ISODate("2026-02-
14T10:10:00Z") },
  { customerId: 8, productId: 113, quantity: 1, paymentMethod: "Cash", amount: 120, date: ISODate("2026-02-14T16:00:00Z")
},
  { customerId: 6, productId: 106, quantity: 1, paymentMethod: "Online", amount: 150, date: ISODate("2026-02-15T11:30:00Z")
},
  { customerId: 2, productId: 102, quantity: 4, paymentMethod: "Credit Card", amount: 100, date: ISODate("2026-02-
15T13:50:00Z") }
]);
```

```
lab06_db> db.transactions.insertMany([
|   { customerId: 1, productId: 101, quantity: 1, paymentMethod: "Credit Card", amount: 1200, date: ISODate("2026-02-01T10:00:00Z") },
|   { customerId: 1, productId: 102, quantity: 2, paymentMethod: "Credit Card", amount: 50, date: ISODate("2026-02-01T11:30:00Z") },
|   { customerId: 2, productId: 104, quantity: 1, paymentMethod: "Cash", amount: 300, date: ISODate("2026-02-01T14:00:00Z") },
|   { customerId: 3, productId: 105, quantity: 10, paymentMethod: "Online", amount: 150, date: ISODate("2026-02-02T09:15:00Z") },
|   { customerId: 4, productId: 110, quantity: 1, paymentMethod: "Credit Card", amount: 500, date: ISODate("2026-02-02T10:00:00Z") },
|   { customerId: 5, productId: 108, quantity: 2, paymentMethod: "Cash", amount: 180, date: ISODate("2026-02-03T12:00:00Z") },
|   { customerId: 6, productId: 111, quantity: 1, paymentMethod: "Credit Card", amount: 200, date: ISODate("2026-02-03T15:45:00Z") },
|   { customerId: 2, productId: 103, quantity: 1, paymentMethod: "Online", amount: 45, date: ISODate("2026-02-04T08:30:00Z") },
|   { customerId: 7, productId: 115, quantity: 3, paymentMethod: "Credit Card", amount: 120, date: ISODate("2026-02-04T11:00:00Z") },
|   { customerId: 8, productId: 109, quantity: 1, paymentMethod: "Cash", amount: 110, date: ISODate("2026-02-05T13:20:00Z") },
|   { customerId: 9, productId: 112, quantity: 1, paymentMethod: "Online", amount: 150, date: ISODate("2026-02-05T16:00:00Z") },
|   { customerId: 10, productId: 107, quantity: 2, paymentMethod: "Credit Card", amount: 160, date: ISODate("2026-02-06T09:00:00Z") },
|   { customerId: 1, productId: 106, quantity: 1, paymentMethod: "Cash", amount: 150, date: ISODate("2026-02-06T14:10:00Z") },
|   { customerId: 3, productId: 114, quantity: 1, paymentMethod: "Online", amount: 60, date: ISODate("2026-02-07T10:30:00Z") },
|   { customerId: 5, productId: 102, quantity: 1, paymentMethod: "Credit Card", amount: 25, date: ISODate("2026-02-07T15:00:00Z") },
|   { customerId: 4, productId: 113, quantity: 1, paymentMethod: "Cash", amount: 120, date: ISODate("2026-02-08T11:45:00Z") },
|   { customerId: 6, productId: 105, quantity: 2, paymentMethod: "Online", amount: 30, date: ISODate("2026-02-08T16:20:00Z") },
|   { customerId: 7, productId: 101, quantity: 1, paymentMethod: "Credit Card", amount: 1200, date: ISODate("2026-02-09T09:15:00Z") },
|   { customerId: 8, productId: 104, quantity: 2, paymentMethod: "Cash", amount: 600, date: ISODate("2026-02-10T12:40:00Z") },
|   { customerId: 2, productId: 110, quantity: 1, paymentMethod: "Online", amount: 500, date: ISODate("2026-02-10T14:50:00Z") },
|   { customerId: 10, productId: 108, quantity: 1, paymentMethod: "Credit Card", amount: 90, date: ISODate("2026-02-11T10:00:00Z") },
|   { customerId: 9, productId: 115, quantity: 1, paymentMethod: "Cash", amount: 40, date: ISODate("2026-02-11T13:15:00Z") },
|   { customerId: 1, productId: 111, quantity: 1, paymentMethod: "Online", amount: 200, date: ISODate("2026-02-12T11:00:00Z") },
|   { customerId: 3, productId: 109, quantity: 2, paymentMethod: "Credit Card", amount: 220, date: ISODate("2026-02-12T15:30:00Z") },
|   { customerId: 5, productId: 107, quantity: 1, paymentMethod: "Cash", amount: 80, date: ISODate("2026-02-13T09:45:00Z") },
|   { customerId: 4, productId: 103, quantity: 2, paymentMethod: "Online", amount: 90, date: ISODate("2026-02-13T14:20:00Z") },
|   { customerId: 7, productId: 114, quantity: 2, paymentMethod: "Credit Card", amount: 120, date: ISODate("2026-02-14T10:10:00Z") },
|   { customerId: 8, productId: 113, quantity: 1, paymentMethod: "Cash", amount: 120, date: ISODate("2026-02-14T16:00:00Z") },
|   { customerId: 6, productId: 106, quantity: 1, paymentMethod: "Online", amount: 150, date: ISODate("2026-02-15T11:30:00Z") },
|   { customerId: 2, productId: 102, quantity: 4, paymentMethod: "Credit Card", amount: 100, date: ISODate("2026-02-15T13:50:00Z") }
| ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a56595977d05ad087c2907'),
    '1': ObjectId('69a56595977d05ad087c2908'),
    '2': ObjectId('69a56595977d05ad087c2909'),
    '3': ObjectId('69a56595977d05ad087c290a'),
    '4': ObjectId('69a56595977d05ad087c290b'),
    '5': ObjectId('69a56595977d05ad087c290c'),
    '6': ObjectId('69a56595977d05ad087c290d'),
    '7': ObjectId('69a56595977d05ad087c290e'),
    '8': ObjectId('69a56595977d05ad087c290f'),
    '9': ObjectId('69a56595977d05ad087c2910'),
    '10': ObjectId('69a56595977d05ad087c2911'),
    '11': ObjectId('69a56595977d05ad087c2912'),
    '12': ObjectId('69a56595977d05ad087c2913'),
    '13': ObjectId('69a56595977d05ad087c2914'),
    '14': ObjectId('69a56595977d05ad087c2915'),
    '15': ObjectId('69a56595977d05ad087c2916'),
    '16': ObjectId('69a56595977d05ad087c2917'),
    '17': ObjectId('69a56595977d05ad087c2918'),
    '18': ObjectId('69a56595977d05ad087c2919'),
    '19': ObjectId('69a56595977d05ad087c291a'),
```

Part A — Aggregation Fundamentals

1. Write an aggregation query that calculates the total revenue generated from all transactions. Your result should produce a single output document showing the computed total.

The goal is to sum the amount field from all documents in the transactions collection to produce a single output document showing the total.

The Query

```
db.transactions.aggregate([
  {
    $group: {
      _id: null,
      totalRevenue: { $sum: "$amount" }
    }
  }
]);
```

Explanation

- **$group**: This stage is used to group documents together.
- **_id: null**: By setting the ID to null, we are telling MongoDB to treat all documents in the collection as a single group rather than splitting them by a specific field.
- **totalRevenue: { $sum: "$amount" }**: This creates a new field called totalRevenue and uses the $sum accumulator to add up the values from the amount field across all transactions.

```
lab06_db> db.transactions.aggregate([
|   {
|     $group: {
|       _id: null,
|       totalRevenue: { $sum: "$amount" }
|     }
|   }
| ]);
[ { _id: null, totalRevenue: 7060 } ]
lab06_db> |
```

2. Write an aggregation pipeline that groups transactions by payment method and counts how many transactions belong to each method. Your output should clearly show the payment method and its transaction count.

For this task, we need to group the transactions based on the paymentMethod and count how many transactions were made using each method.

The Query

```
db.transactions.aggregate([
  {
    $group: {
      _id: "$paymentMethod",
      transactionCount: { $sum: 1 }
    }
  }
]);
```

Explanation

- **_id: "$paymentMethod"**: This tells MongoDB to create a group for every unique value found in the paymentMethod field.
- **transactionCount: { $sum: 1 }**: This is a standard way to count documents in a group. For every document that falls into a specific payment group, it adds 1 to the total.

```
lab06_db> db.transactions.aggregate([
|    {
|      $group: {
|        _id: "$paymentMethod",
|        transactionCount: { $sum: 1 }
|      }
|    }
| ]);
[
  { _id: 'Cash', transactionCount: 9 },
  { _id: 'Credit Card', transactionCount: 12 },
  { _id: 'Online', transactionCount: 9 }
]
lab06_db> |
```

3. Write an aggregation query that computes the average transaction amount across the entire transactions collection. Ensure that only the amount field is used for the calculation.

This task requires calculating the average amount across all transactions using only the amount field.

The Query

```
db.transactions.aggregate([
  {
    $group: {
      _id: null,
      averageAmount: { $avg: "$amount" }
    }
  }
```

```
]);
```

Explanation

- **_id: null**: Similar to Task 1, we use null because we want to calculate the average for the entire collection, not split into groups.
- **$avg: "$amount"**: The $avg operator automatically sums the amount values and divides by the total number of documents.

```
lab06_db> db.transactions.aggregate([
|   {
|     $group: {
|       _id: null,
|       averageAmount: { $avg: "$amount" }
|     }
|   }
| ]);
[ { _id: null, averageAmount: 235.33333333333334 } ]
lab06_db> |
```

Part B - Grouping and Statistical Summaries

4. Create an aggregation pipeline that groups transactions by customerId and calculates the total amount spent by each customer. Also compute how many transactions each customer has made and their average transaction value. Sort the final result from highest spender to lowest spender.

   We need to group transactions by customerId to calculate the **total amount spent**, the **number of transactions**, and the **average transaction value** for each customer. Finally, the results must be sorted from the highest spender to the lowest.

The Query

```
db.transactions.aggregate([
  {
    $group: {
      _id: "$customerId",
```

```
      totalSpent: { $sum: "$amount" },
      transactionCount: { $sum: 1 },
      averageTransactionValue: { $avg: "$amount" }
    }
  },
  {
    $sort: { totalSpent: -1 }
  }
]);
```

Explanation

- **$group Stage**: Collects all transactions sharing the same customerId.
    - totalSpent: Uses $sum on the amount field to get the total expenditure per person.
    - transactionCount: Uses { $sum: 1 } to count how many documents exist for that customer.
    - averageTransactionValue: Uses $avg on the amount field to find the mean value per transaction.
- **$sort Stage**: Takes the grouped results and organizes them by totalSpent in descending order (-1), identifying the "heavy users" or top spenders first.

```
lab06_db> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpent: { $sum: "$amount" },
|       transactionCount: { $sum: 1 },
|       averageTransactionValue: { $avg: "$amount" }
|     }
|   },
|   {
|     $sort: { totalSpent: -1 }
|   }
| ]);
|
[
  {
    _id: 1,
    totalSpent: 1600,
    transactionCount: 4,
    averageTransactionValue: 400
  },
  {
    _id: 7,
    totalSpent: 1440,
    transactionCount: 3,
    averageTransactionValue: 480
  },
  {
    _id: 2,
    totalSpent: 945,
    transactionCount: 4,
    averageTransactionValue: 236.25
  },
  {
    _id: 8,
    totalSpent: 830,
    transactionCount: 3,
    averageTransactionValue: 276.6666666666667
  },
  {
    _id: 4,
    totalSpent: 710,
    transactionCount: 3,
    averageTransactionValue: 236.66666666666666
  },
  {
    _id: 3,
    totalSpent: 430,
    transactionCount: 3,
    averageTransactionValue: 143.33333333333334
  },
  {
    _id: 6,
    totalSpent: 380,
    transactionCount: 3,
    averageTransactionValue: 126.66666666666667
```

```
  {
    _id: 5,
    totalSpent: 285,
    transactionCount: 3,
    averageTransactionValue: 95
  },
  {
    _id: 10,
    totalSpent: 250,
    transactionCount: 2,
    averageTransactionValue: 125
  },
  {
    _id: 9,
    totalSpent: 190,
    transactionCount: 2,
    averageTransactionValue: 95
  }
]
```

5. Build an aggregation query that groups transactions by productId and calculates total quantity sold and total revenue per product. Also compute the average quantity per transaction for each product.

We will analyze the performance of each product. We need to group transactions by productId to calculate the **total quantity sold**, **total revenue generated**, and the **average quantity per transaction** for every product.

The Query

```
db.transactions.aggregate([
  {
    $group: {
      _id: "$productId",
```

```
      totalQuantitySold: { $sum: "$quantity" },
      totalRevenue: { $sum: "$amount" },
      averageQuantityPerOrder: { $avg: "$quantity" }
    }
  }
]);
```

Explanation

- **$group Stage**: Aggregates all transaction records that share the same productId.
- **totalQuantitySold: { $sum: "$quantity" }**: Sums the quantity field for each product to show how many units were sold in total.
- **totalRevenue: { $sum: "$amount" }**: Sums the amount field to calculate the gross income for each specific product.
- **averageQuantityPerOrder: { $avg: "$quantity" }**: Calculates the mean number of units purchased in a single transaction for that product.

```
lab06_db> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$productId",
|       totalQuantitySold: { $sum: "$quantity" },
|       totalRevenue: { $sum: "$amount" },
|       averageQuantityPerOrder: { $avg: "$quantity" }
|     }
|   }
| ]);
|
[
  {
    _id: 103,
    totalQuantitySold: 3,
    totalRevenue: 135,
    averageQuantityPerOrder: 1.5
  },
  {
    _id: 110,
    totalQuantitySold: 2,
    totalRevenue: 1000,
    averageQuantityPerOrder: 1
  },
  {
    _id: 101,
    totalQuantitySold: 2,
    totalRevenue: 2400,
    averageQuantityPerOrder: 1
  },
  {
    _id: 102,
    totalQuantitySold: 7,
    totalRevenue: 175,
    averageQuantityPerOrder: 2.3333333333333335
  },
  {
    _id: 105,
    totalQuantitySold: 12,
    totalRevenue: 180,
    averageQuantityPerOrder: 6
  },
  {
    _id: 112,
    totalQuantitySold: 1,
    totalRevenue: 150,
    averageQuantityPerOrder: 1
  },
  {
    _id: 115,
    totalQuantitySold: 4,
    totalRevenue: 160,
    averageQuantityPerOrder: 2
  },
  {
    _id: 107,
    totalQuantitySold: 3,
    totalRevenue: 240,
    averageQuantityPerOrder: 1.5
  },
  {
    _id: 106,
    totalQuantitySold: 2,
    totalRevenue: 300,
```

```
    averageQuantityPerOrder: 1
  },
  {
    _id: 114,
    totalQuantitySold: 3,
    totalRevenue: 180,
    averageQuantityPerOrder: 1.5
  },
  {
    _id: 113,
    totalQuantitySold: 2,
    totalRevenue: 240,
    averageQuantityPerOrder: 1
  },
  {
    _id: 104,
    totalQuantitySold: 3,
    totalRevenue: 900,
    averageQuantityPerOrder: 1.5
  },
  {
    _id: 108,
    totalQuantitySold: 3,
    totalRevenue: 270,
    averageQuantityPerOrder: 1.5
  },
  {
    _id: 111,
    totalQuantitySold: 2,
    totalRevenue: 400,
    averageQuantityPerOrder: 1
  },
  {
    _id: 109,
    totalQuantitySold: 3,
    totalRevenue: 330,
    averageQuantityPerOrder: 1.5
  }
]
lab06_db> |
```

6. Construct a pipeline that first filters transactions to include only those paid by credit card and with amount greater than 5. Then compute the total revenue from only those filtered transactions. Make sure your pipeline clearly separates the filtering stage and the grouping stage.

For this task, we must construct a pipeline that specifically filters transactions to include only those paid by **"Credit Card"** and with an amount **greater than 5**. After filtering, we must

compute the total revenue from only those specific transactions. The pipeline must clearly separate the filtering stage and the grouping stage.

The Query

```
db.transactions.aggregate([
  {
    $match: {
      paymentMethod: "Credit Card",
      amount: { $gt: 5 }
    }
  },
  {
    $group: {
      _id: null,
      totalFilteredRevenue: { $sum: "$amount" }
    }
  }
]);
```

Explanation

- **$match Stage**: This is the filtering stage. It selects only the documents where the paymentMethod is exactly "Credit Card" and the amount is greater than 5 using the $gt (greater than) operator.
- **$group Stage**: This is the grouping stage. Since we want a single total for all filtered transactions, we use _id: null and apply the $sum accumulator to the amount field.
- **Separation of Concerns**: By placing $match before $group, we ensure the database only processes relevant records, which is more efficient than grouping first and filtering later

```
lab06_db> db.transactions.aggregate([
|   {
|     $match: {
|       paymentMethod: "Credit Card",
|       amount: { $gt: 5 }
|     }
|   },
|   {
|     $group: {
|       _id: null,
|       totalFilteredRevenue: { $sum: "$amount" }
|     }
|   }
| ]);
|
[ { _id: null, totalFilteredRevenue: 3985 } ]
lab06_db>
```

Part C — Multi Stage Aggregation Pipelines

7. Write a multi stage aggregation pipeline that produces a daily revenue report. First extract or format the transaction date, then group transactions by date, and finally compute total revenue per day. Sort the output by date in ascending order.

We need to produce a report that groups transactions by their specific date, calculates the total revenue for that day, and sorts the output chronologically.

The Query

```
db.transactions.aggregate([
  {
    $project: {
      dateOnly: { $dateToString: { format: "%Y-%m-%d", date: "$date" } },
      amount: 1
    }
  },
  {
    $group: {
      _id: "$dateOnly",
      dailyTotal: { $sum: "$amount" }
    }
  },
  {
    $sort: { _id: 1 }
  }
]);
```

Explanation

- **$project Stage**: Since the date field contains time information, we use $dateToString to extract only the "Year-Month-Day" portion.
- **$group Stage**: Groups all transactions that occurred on the same formatted date string and sums their amount.
- **$sort Stage**: Orders the final list by the date (_id) in ascending order (1) to show the revenue trend over time.

```
lab06_db> db.transactions.aggregate([
|   {
|     $project: {
|       dateOnly: { $dateToString: { format: "%Y-%m-%d", date: "$date" } },
|       amount: 1
|     }
|   },
|   {
|     $group: {
|       _id: "$dateOnly",
|       dailyTotal: { $sum: "$amount" }
|     }
|   },
|   {
|     $sort: { _id: 1 }
|   }
| ]);
|
[
  { _id: '2026-02-01', dailyTotal: 1550 },
  { _id: '2026-02-02', dailyTotal: 650 },
  { _id: '2026-02-03', dailyTotal: 380 },
  { _id: '2026-02-04', dailyTotal: 165 },
  { _id: '2026-02-05', dailyTotal: 260 },
  { _id: '2026-02-06', dailyTotal: 310 },
  { _id: '2026-02-07', dailyTotal: 85 },
  { _id: '2026-02-08', dailyTotal: 150 },
  { _id: '2026-02-09', dailyTotal: 1200 },
  { _id: '2026-02-10', dailyTotal: 1100 },
  { _id: '2026-02-11', dailyTotal: 130 },
  { _id: '2026-02-12', dailyTotal: 420 },
  { _id: '2026-02-13', dailyTotal: 170 },
  { _id: '2026-02-14', dailyTotal: 240 },
  { _id: '2026-02-15', dailyTotal: 250 }
]
lab06_db> |
```

8. Develop an aggregation pipeline that identifies the top five customers by total spending. Your pipeline should group by customer, compute total spending, sort in descending order, and then limit the output to five records.

This pipeline identifies the highest spenders by grouping by customer, sorting the totals, and then restricting the output.

The Query

```
db.transactions.aggregate([
  {
    $group: {
      _id: "$customerId",
      totalSpending: { $sum: "$amount" }
    }
  },
  {
    $sort: { totalSpending: -1 }
  },
  {
    $limit: 5
  }
]);
```

Explanation

- **$group Stage**: Aggregates the total spending for every unique customerId.
- **$sort Stage**: Sorts the resulting documents by totalSpending in descending order (-1) so the biggest spenders are at the top.
- **$limit Stage**: Restricts the final output to only the first 5 records, effectively creating a "Top 5" leaderboard.

```
lab06_db> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: "$amount" }
|     }
|   },
|   {
|     $sort: { totalSpending: -1 }
|   },
|   {
|     $limit: 5
|   }
| ]);
|
[
  { _id: 1, totalSpending: 1600 },
  { _id: 7, totalSpending: 1440 },
  { _id: 2, totalSpending: 945 },
  { _id: 8, totalSpending: 830 },
  { _id: 4, totalSpending: 710 }
]
lab06_db> |
```

Part D — Joining Collections

9. Write an aggregation pipeline that joins transactions with the customers collection so that each transaction output includes the customer's first and last name along with the transaction amount and payment method.

We need to pull the customer's first and last name into each transaction record.

The Query

```
db.transactions.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "customerId",
      foreignField: "_id",
      as: "customer_details"
    }
  },
  {
    $unwind: "$customer_details"
  },
```

```
  {
    $project: {
      _id: 1,
      amount: 1,
      paymentMethod: 1,
      firstName: "$customer_details.firstName",
      lastName: "$customer_details.lastName"
    }
  }
]);
```

Explanation

- **$lookup**: Matches the customerId from the transactions collection to the _id in the customers collection
- **$unwind**: Deconstructs the customer_details array (created by lookup) into a single object for easier access.
- **$project**: Reshapes the output to show only the transaction ID, amount, payment method, and the joined name fields.

```
lab06_db> db.transactions.aggregate([
|   {
|     $lookup: {
|       from: "customers",
|       localField: "customerId",
|       foreignField: "_id",
|       as: "customer_details"
|     }
|   },
|   {
|     $unwind: "$customer_details"
|   },
|   {
|     $project: {
|       _id: 1,
|       amount: 1,
|       paymentMethod: 1,
|       firstName: "$customer_details.firstName",
|       lastName: "$customer_details.lastName"
|     }
|   }
| ]);
|
[
  {
    _id: ObjectId('69a56595977d05ad087c2907'),
    paymentMethod: 'Credit Card',
    amount: 1200,
    firstName: 'John',
    lastName: 'Doe'
  },
  {
    _id: ObjectId('69a56595977d05ad087c2908'),
    paymentMethod: 'Credit Card',
    amount: 50,
    firstName: 'John',
    lastName: 'Doe'
  },
  {
    _id: ObjectId('69a56595977d05ad087c2909'),
    paymentMethod: 'Cash',
    amount: 300,
    firstName: 'Jane',
    lastName: 'Smith'
  },
  {
    _id: ObjectId('69a56595977d05ad087c290a'),
    paymentMethod: 'Online',
    amount: 150,
    firstName: 'Ali',
    lastName: 'Khan'
  },
  {
    _id: ObjectId('69a56595977d05ad087c290b'),
    paymentMethod: 'Credit Card',
    amount: 500,
    firstName: 'Maria',
    lastName: 'Garcia'
  },
  {
    _id: ObjectId('69a56595977d05ad087c290c'),
    paymentMethod: 'Cash',
```

10. Write another aggregation pipeline that joins transactions with the products collection and displays the product name together with the quantity and amount for each transaction.

This task displays the product name alongside the quantity and amount for each transaction.

The Query

```
db.transactions.aggregate([
  {
    $lookup: {
      from: "products",
```

```
      localField: "productId",
      foreignField: "_id",
      as: "product_details"
    }
  },
  {
    $unwind: "$product_details"
  },
  {
    $project: {
      _id: 1,
      productName: "$product_details.name",
      quantity: 1,
      amount: 1
    }
  }
]);
```

Explanation

- **$lookup**: Joins the products collection based on the productId.
- **$project**: Extracts the name from the joined product data and renames it to productName for clarity.

```
lab06_db> db.transactions.aggregate([
|   {
|     $lookup: {
|       from: "products",
|       localField: "productId",
|       foreignField: "_id",
|       as: "product_details"
|     }
|   },
|   {
|     $unwind: "$product_details"
|   },
|   {
|     $project: {
|       _id: 1,
|       productName: "$product_details.name",
|       quantity: 1,
|       amount: 1
|     }
|   }
| ]);
|
[
  {
    _id: ObjectId('69a56595977d05ad087c2907'),
    quantity: 1,
    amount: 1200,
    productName: 'Laptop'
  },
  {
    _id: ObjectId('69a56595977d05ad087c2908'),
    quantity: 2,
    amount: 50,
    productName: 'Mouse'
  },
  {
    _id: ObjectId('69a56595977d05ad087c2909'),
    quantity: 1,
    amount: 300,
    productName: 'Monitor'
  },
  {
    _id: ObjectId('69a56595977d05ad087c290a'),
    quantity: 10,
    amount: 150,
    productName: 'USB-C Cable'
  },
  {
    _id: ObjectId('69a56595977d05ad087c290b'),
    quantity: 1,
    amount: 500,
    productName: 'GPU'
  },
  {
    _id: ObjectId('69a56595977d05ad087c290c'),
    quantity: 2,
    amount: 180,
    productName: 'RAM 16GB'
  },
  {
    _id: ObjectId('69a56595977d05ad087c290d'),
    quantity: 1,
    amount: 200,
    productName: 'Chair'
```

11. Build a full analytical pipeline that joins transactions with both customers and products. Your final output should show customer name, product name, quantity, payment method, and amount for each record.

This builds a full report showing the customer name, product name, quantity, payment method, and amount.

The Query

```
db.transactions.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "customerId",
      foreignField: "_id",
      as: "cust"
    }
  },
  { $unwind: "$cust" },
  {
    $lookup: {
      from: "products",
      localField: "productId",
      foreignField: "_id",
      as: "prod"
    }
  },
  { $unwind: "$prod" },
  {
    $project: {
      _id: 0,
      customerName: { $concat: ["$cust.firstName", " ", "$cust.lastName"] },
      productName: "$prod.name",
      quantity: 1,
      paymentMethod: 1,
      amount: 1
    }
  }
]);
```

Explanation

- **Double $lookup**: This pipeline demonstrates how to join three collections by chaining lookup stages.

- **$concat**: Combines the first and last name into a single customerName string for a cleaner report.
- **Final Output**: This provides a complete "flat" view of the transaction data, similar to a SQL Join across three tables.

```
lab06_db> db.transactions.aggregate([
|   {
|     $lookup: {
|       from: "customers",
|       localField: "customerId",
|       foreignField: "_id",
|       as: "cust"
|     }
|   },
|   { $unwind: "$cust" },
|   {
|     $lookup: {
|       from: "products",
|       localField: "productId",
|       foreignField: "_id",
|       as: "prod"
|     }
|   },
|   { $unwind: "$prod" },
|   {
|     $project: {
|       _id: 0,
|       customerName: { $concat: ["$cust.firstName", " ", "$cust.lastName"] },
|       productName: "$prod.name",
|       quantity: 1,
|       paymentMethod: 1,
|       amount: 1
|     }
|   }
| ]);
|
[
  {
    quantity: 1,
    paymentMethod: 'Credit Card',
    amount: 1200,
    customerName: 'John Doe',
    productName: 'Laptop'
  },
  {
    quantity: 2,
    paymentMethod: 'Credit Card',
    amount: 50,
    customerName: 'John Doe',
    productName: 'Mouse'
  },
  {
    quantity: 1,
    paymentMethod: 'Cash',
    amount: 300,
    customerName: 'Jane Smith',
    productName: 'Monitor'
  },
  {
    quantity: 10,
    paymentMethod: 'Online',
    amount: 150,
    customerName: 'Ali Khan',
    productName: 'USB-C Cable'
  },
  {
    quantity: 1,
    paymentMethod: 'Credit Card',
```

Part E — Derived Fields and Categorization

12. Create an aggregation pipeline that computes total revenue per product and sorts the products from highest revenue to lowest revenue to produce a ranking.

This task requires calculating the total revenue per product and sorting the results from highest to lowest to create a clear ranking.

The Query

```
db.transactions.aggregate([
  {
    $group: {
      _id: "$productId",
      totalRevenue: { $sum: "$amount" }
    }
  },
  {
    $sort: { totalRevenue: -1 }
  }
]);
```

Explanation

- **$group Stage**: Aggregates all transactions by productId and uses the $sum accumulator on the amount field to find the total earnings for each item.
- **$sort Stage**: Orders the resulting documents by totalRevenue in descending order (-1) to identify the top-performing products.
- category while hiding the default _id field.

```
lab06_db> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$productId",
|       totalRevenue: { $sum: "$amount" }
|     }
|   },
|   {
|     $sort: { totalRevenue: -1 }
|   }
| ]);
|
[
  { _id: 101, totalRevenue: 2400 },
  { _id: 110, totalRevenue: 1000 },
  { _id: 104, totalRevenue: 900 },
  { _id: 111, totalRevenue: 400 },
  { _id: 109, totalRevenue: 330 },
  { _id: 106, totalRevenue: 300 },
  { _id: 108, totalRevenue: 270 },
  { _id: 107, totalRevenue: 240 },
  { _id: 113, totalRevenue: 240 },
  { _id: 114, totalRevenue: 180 },
  { _id: 105, totalRevenue: 180 },
  { _id: 102, totalRevenue: 175 },
  { _id: 115, totalRevenue: 160 },
  { _id: 112, totalRevenue: 150 },
  { _id: 103, totalRevenue: 135 }
]
lab06_db> |
```

13. Write an aggregation pipeline that first computes total spending per customer and then assigns each customer into a spending category labeled High, Medium, or Low based on their total spending. Clearly show the customerId, total spending, and assigned category.

We must compute the total spending per customer and then assign them into a category **High**, **Medium**, or **Low** based on their total spending.

The Query

Note: Adjusted the range values to 500 and 1000 to show all categories

```
db.transactions.aggregate([
  {
    $group: {
      _id: "$customerId",
      totalSpending: { $sum: "$amount" }
    }
  },
  {
    $project: {
      customerId: "$_id",
      totalSpending: 1,
      category: {
        $switch: {
          branches: [
            { case: { $gte: ["$totalSpending", 1000] }, then: "High" },
            { case: { $and: [{ $gte: ["$totalSpending", 500] }, { $lt:
["$totalSpending", 1000] }] }, then: "Medium" }
          ],
          default: "Low"
        }
      },
      _id: 0
    }
  }
]);
```

Explanation

- **$group Stage**: Calculates the total lifetime value for each customer.
- **$switch Operator**: Evaluates the totalSpending against specific thresholds to assign a text label (High, Medium, or Low).
- **$project Stage**: Clearly displays the customerId, totalSpending, and the newly assigned

```
lab06_db>
| db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: "$amount" }
|     }
|   },
|   {
|     $project: {
|       customerId: "$_id",
|       totalSpending: 1,
|       category: {
|         $switch: {
|           branches: [
|             { case: { $gte: ["$totalSpending", 1000] }, then: "High" },
|             { case: { $and: [{ $gte: ["$totalSpending", 500] }, { $lt: ["$totalSpending", 1000] }] }, then: "Medium" }
|           ],
|           default: "Low"
|         }
|       },
|       _id: 0
|     }
|   }
| ]);
|
[
  { totalSpending: 830, customerId: 8, category: 'Medium' },
  { totalSpending: 380, customerId: 6, category: 'Low' },
  { totalSpending: 945, customerId: 2, category: 'Medium' },
  { totalSpending: 1600, customerId: 1, category: 'High' },
  { totalSpending: 430, customerId: 3, category: 'Low' },
  { totalSpending: 285, customerId: 5, category: 'Low' },
  { totalSpending: 1440, customerId: 7, category: 'High' },
  { totalSpending: 710, customerId: 4, category: 'Medium' },
  { totalSpending: 190, customerId: 9, category: 'Low' },
  { totalSpending: 250, customerId: 10, category: 'Low' }
]
lab06_db>
```

Part F — MapReduce Processing

14. Implement a MapReduce job that calculates total revenue per product. Your map function must emit productId and amount, and your reduce function must sum the values. Store results in a collection named mr_product_revenue.

This task calculates revenue by product and stores the results in a new collection named mr_product_revenue.

The Query

```
var mapProduct = function() {
    emit(this.productId, this.amount); // Emit productId as key, amount as value
};

var reduceProduct = function(key, values) {
    return Array.sum(values); // Sum all amounts for the same productId
};

db.transactions.mapReduce(
    mapProduct,
    reduceProduct,
    { out: "mr_product_revenue" }
```

```
);

// To view results
db.mr_product_revenue.find().pretty();
```

Explanation

Map Function: Iterates through the transactions and emits the productId as the key and the amount as the value.

Reduce Function: Receives an array of all amounts for a specific product and uses Array.sum() to calculate the total.

Output: The results are saved into mr_product_revenue rather than just being displayed in the console.

```
lab06_db> var mapProduct = function() {
|     emit(this.productId, this.amount); // Emit productId as key, amount
| };

lab06_db> var reduceProduct = function(key, values) {
|     return Array.sum(values); // Sum all amounts for the same productId
| };

lab06_db> db.transactions.mapReduce(
|     mapProduct,
|     reduceProduct,
|     { out: "mr_product_revenue" }
| );
DeprecationWarning: Collection.mapReduce() is deprecated. Use an aggregat
See https://mongodb.com/docs/manual/core/map-reduce for details.
{ result: 'mr_product_revenue', ok: 1 }
lab06_db> db.mr_product_revenue.find().pretty();
[
  { _id: 103, value: 135 },
  { _id: 113, value: 240 },
  { _id: 104, value: 900 },
  { _id: 101, value: 2400 },
  { _id: 105, value: 180 },
  { _id: 115, value: 160 },
  { _id: 109, value: 330 },
  { _id: 112, value: 150 },
  { _id: 106, value: 300 },
  { _id: 111, value: 400 },
  { _id: 102, value: 175 },
  { _id: 110, value: 1000 },
  { _id: 108, value: 270 },
  { _id: 114, value: 180 },
  { _id: 107, value: 240 }
]
lab06_db> |
```

15. Implement a MapReduce job that computes total spending per customer and store the output in mr_customer_spending.

This job computes the total amount spent by each customer and stores it in mr_customer_spending.

The Query

```
var mapCustomer = function() {
    emit(this.customerId, this.amount);
};

var reduceCustomer = function(key, values) {
    return Array.sum(values);
};

db.transactions.mapReduce(
    mapCustomer,
    reduceCustomer,
    { out: "mr_customer_spending" }
);

db.mr_customer_spending.find().pretty();
```

Explanation

- **Map Function**: This function iterates through each document in the transactions collection and uses emit(this.customerId, this.amount) to create a key-value pair where the Customer ID is the key and the transaction amount is the value.
- **Reduce Function**: MongoDB groups all amounts associated with the same customerId. The reduce function then receives this array of values and uses Array.sum(values) to calculate the total lifetime spending for that specific customer.
- **Output**: The final results, consisting of unique Customer IDs and their corresponding total spending, are persisted into a new collection called mr_customer_spending.

```
lab06_db> var mapCustomer = function() {
|       emit(this.customerId, this.amount);
| };
|
| var reduceCustomer = function(key, values) {
|       return Array.sum(values);
| };
|
| db.transactions.mapReduce(
|       mapCustomer,
|       reduceCustomer,
|       { out: "mr_customer_spending" }
| );
|
| db.mr_customer_spending.find().pretty();
|
[
  { _id: 8, value: 830 },
  { _id: 7, value: 1440 },
  { _id: 2, value: 945 },
  { _id: 1, value: 1600 },
  { _id: 3, value: 430 },
  { _id: 5, value: 285 },
  { _id: 6, value: 380 },
  { _id: 4, value: 710 },
  { _id: 10, value: 250 },
  { _id: 9, value: 190 }
]
lab06_db> |
```

16. Implement a MapReduce job that computes total quantity sold per product using the quantity field.

This task tracks how many units of each product were sold using the quantity field.

The Query

```
var mapQty = function() {
    emit(this.productId, this.quantity); // Emit quantity instead of amount
};

var reduceQty = function(key, values) {
    return Array.sum(values);
};

db.transactions.mapReduce(
    mapQty,
    reduceQty,
    { out: "mr_product_quantity" }
);

db.mr_product_quantity.find().pretty();
```

Explanation

Map Function: In this job, the map function focuses on inventory tracking. It emits the productId as the key and the quantity field as the value for every transaction.

Reduce Function: The reducer aggregates the quantities for each unique product key. By summing these values, it computes the total volume of stock sold for every item in the products collection.

Workflow: This demonstrates MapReduce's flexibility; by simply changing the value emitted in the Map phase (from amount to quantity), we can generate a completely different analytical report.

```
lab06_db> var mapQty = function() {
|     emit(this.productId, this.quantity); // Emit quantity instead of amount
| };
|
| var reduceQty = function(key, values) {
|     return Array.sum(values);
| };
|
| db.transactions.mapReduce(
|     mapQty,
|     reduceQty,
|     { out: "mr_product_quantity" }
| );
|
| db.mr_product_quantity.find().pretty();
|
[
  { _id: 103, value: 3 },
  { _id: 113, value: 2 },
  { _id: 104, value: 3 },
  { _id: 101, value: 2 },
  { _id: 105, value: 12 },
  { _id: 115, value: 4 },
  { _id: 109, value: 3 },
  { _id: 112, value: 1 },
  { _id: 106, value: 2 },
  { _id: 111, value: 2 },
  { _id: 102, value: 7 },
  { _id: 110, value: 2 },
  { _id: 108, value: 3 },
  { _id: 114, value: 3 },
  { _id: 107, value: 3 }
]
lab06_db> |
```

17. Implement a MapReduce job that computes monthly revenue by extracting year and month from transactionDate and emitting a combined key.

This requires extracting the Year and Month from the transactionDate to emit a combined key.

The Query

```
var mapMonthly = function() {
    var date = new Date(this.date);
    var key = date.getFullYear() + "-" + (date.getMonth() + 1); // Combine Year
and Month
    emit(key, this.amount);
};

var reduceMonthly = function(key, values) {
    return Array.sum(values);
};

db.transactions.mapReduce(
    mapMonthly,
    reduceMonthly,
    { out: "mr_monthly_revenue" }
);

db.mr_monthly_revenue.find().pretty();
```

Explanation

- **Complex Key Mapping**: Unlike previous tasks that used existing fields as keys, this map function performs data transformation. It extracts the **Year** and **Month** from the transactionDate to construct a custom string key.
- **Reduce Function**: The reducer sums all transaction amounts that fall within the same Year-Month key.
- **Analytical Value**: This process allows the system to condense 30+ individual daily transactions into a high-level monthly financial summary, which is essential for long-term business trend analysis

```
lab06_db> var mapMonthly = function() {
|     var date = new Date(this.date);
|     var key = date.getFullYear() + "-" + (date.getMonth() + 1); // Combine Year and Month
|     emit(key, this.amount);
| };
|
| var reduceMonthly = function(key, values) {
|     return Array.sum(values);
| };
|
| db.transactions.mapReduce(
|     mapMonthly,
|     reduceMonthly,
|     { out: "mr_monthly_revenue" }
| );
|
| db.mr_monthly_revenue.find().pretty();
|
[ { _id: '2026-2', value: 7060 } ]
lab06_db> |
```

Part G — Aggregation vs MapReduce Comparison

18. Compute revenue per product using both an aggregation pipeline and a MapReduce job. After executing both methods, write a short explanation comparing them in terms of readability, complexity, and observed execution behavior.

1. The Aggregation Pipeline Query

```
db.transactions.aggregate([
  {
    $group: {
      _id: "$productId",
      totalRevenue: { $sum: "$amount" }
    }
  }
]);
```

```
lab06_db> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$productId",
|       totalRevenue: { $sum: "$amount" }
|     }
|   }
| ]);
|
[
  { _id: 101, totalRevenue: 2400 },
  { _id: 102, totalRevenue: 175 },
  { _id: 112, totalRevenue: 150 },
  { _id: 107, totalRevenue: 240 },
  { _id: 106, totalRevenue: 300 },
  { _id: 113, totalRevenue: 240 },
  { _id: 115, totalRevenue: 160 },
  { _id: 114, totalRevenue: 180 },
  { _id: 105, totalRevenue: 180 },
  { _id: 104, totalRevenue: 900 },
  { _id: 110, totalRevenue: 1000 },
  { _id: 108, totalRevenue: 270 },
  { _id: 111, totalRevenue: 400 },
  { _id: 109, totalRevenue: 330 },
  { _id: 103, totalRevenue: 135 }
]
lab06_db> |
```

2. The MapReduce Job Query

```
var mapProduct = function() {
    emit(this.productId, this.amount);
};

var reduceProduct = function(key, values) {
    return Array.sum(values);
};

db.transactions.mapReduce(
    mapProduct,
    reduceProduct,
    { out: "mr_comparison_results" }
);

db.mr_comparison_results.find().pretty();
```

```
lab06_db> var mapProduct = function() {
|     emit(this.productId, this.amount);
| };
|
| var reduceProduct = function(key, values) {
|     return Array.sum(values);
| };
|
| db.transactions.mapReduce(
|     mapProduct,
|     reduceProduct,
|     { out: "mr_comparison_results" }
| );
|
| db.mr_comparison_results.find().pretty();
|
[
  { _id: 101, value: 2400 },
  { _id: 110, value: 1000 },
  { _id: 108, value: 270 },
  { _id: 111, value: 400 },
  { _id: 115, value: 160 },
  { _id: 112, value: 150 },
  { _id: 114, value: 180 },
  { _id: 109, value: 330 },
  { _id: 105, value: 180 },
  { _id: 104, value: 900 },
  { _id: 102, value: 175 },
  { _id: 113, value: 240 },
  { _id: 107, value: 240 },
  { _id: 106, value: 300 },
  { _id: 103, value: 135 }
]
```

3. Comparison and Analysis

Feature	*Aggregation Framework*	*MapReduce*
Readability	High - uses a declarative, stage-based JSON syntax.	Lower - requires writing custom JavaScript functions.
Complexity	Simple for standard math (sum, avg, count).	More complex; better for highly custom logic.
Performance	Generally faster; runs natively in C++ within the server.	Generally slower; involves a JavaScript execution engine.
Output	Returns a cursor of results directly.	Typically outputs to a new collection.